AMATH 482: HOMEWORK 2

DORIAN RAMEY

University of Washington, Seattle, WA
dramey2@uw.edu

Abstract. We developed an algorithm based on principle component analysis to project data recording the movement of a humanoid robot, OptimuS-VD, into lower dimensions in order to classify movements as either jumping, running, or walking. This algorithm could now be applied to the robot's measurements in real time to classify them with approximately 80 percent accuracy for k-modes between 2 and 6, and 93.3 percent accuracy for a k-mode of 7, which may be higher depending on the sample.

1. Introduction and Overview

In this project, we attempt to classify the movements of a humanoid robot, OptimuS-VD, as jumping, running, or walking. The robot has built in sensors that record the movements of 38 of its joints as Euler angles, or matrices representing a movement around the y-, x-, and z-axes [5]. We have 5, 1.4 second samples of each of the movement patterns. Our goal is to build an algorithm that classifies OptimuS-VD's movements in real time. To develop this algorithm, we must project the training data into a lower dimension and visualize it.

2. Theoretical Background

2.1. Singular Value Decomposition.

Theorem 2.1 (Singular Value Decomposition Theory)**.** *Let $A \in \mathbb{R}^{mxn}$, there exists unitary matrices $U \in \mathbb{R}^{mxm}$, $V \in \mathbb{R}^{nxn}$, and a diagonal matrix $\Sigma \in \mathbb{R}^{mxn}$ with non-negative entries for which $A = U\Sigma V^T$*

[1]

The singular value decomposition (SVD) of a matrix is a matrix factorization that elucidates: 1) axes, or left singular vectors (U), 2) magnitude of axes, or singular values (Σ), 3) original axes on unit sphere that map to the axes in 1 (V) of a shape under a matrix

2.2. Principle Component Analysis.

Principle Component Analysis (PCA) is a method for reducing the dimensionality of the data. To perform PCA, we first must center the data using the equation: $Z_i = X_i - \mu$ where μ represents the mean of x0 through xn. [2]Centering the data keeps the mean from impacting our results. If the mean is not subtracted, the top PCA modes are likely the ones that are highly correlated with the mean which, for our purposes, is not very informative.[3] This centered data is then put into a matrix, Z. We use the SVD as described above 2.1 to factorize the matrix into its components.

To reduce the dimensionality, we pick a target dimensionality k, then truncate the singular vectors to only those that correspond to the top k singular values. This gives us U^k which allows us to project samples onto the k most dominant directions in the original data by applying $(U^k)^T x$[2]

Date: February 12, 2026.

3. Algorithm Implementation and Development

3.0.1. *PCA on Full Dataset.* Our first step in solving the problem outlined in the Introduction 1 is to load all of the training data into a 100 x 1500 training matrix. Once the data is compiled, we can apply PCA as described in the Principle Component Analysis section above 2.2. To do this, we use the PCA function of the sklearn library [4] on the full training data.

3.0.2. *Apply PCA to Each Sample and Plot.* Next, we find the projection of each sample in reduced dimensionality. We take k = 2 and k = 3 so that we can visualize our results. For each of these k-values, we truncate the U vector calculated by the PCA method to only the singular vectors corresponding to the top k singular values and apply Equation 1 2.2 stated in the Principle Component Analysis section 2.2. Each sample is projected individually and all samples are plotted together to visualize each movement in 2 and 3 dimensions.

3.0.3. *Calculate Centroids and Means for Each Movement.* For each movement, we must then calculate the centroid which will give us a value to compare future samples to in order to classify them. To calculate the centroid, we simply find the mean of each movement in k-modes in PCA space and save these values in an array for future comparisons.

3.0.4. *Calculate Distance Between Each Sample's Centroid and Movement Means and Classify Samples.* We then attempt to classify the training samples by finding each samples distance from each of the three movement centroids in a chosen k-mode. The sample will be classified as whichever movement's centroid it is closest to. Once all samples are classified, we compare the classification to the known true movement to find the accuracy percentage. This allows us to compare various k-values and choose one with higher accuracy. We perform this not only for the k-values of 2 and 3 that we visualized before, but other k-values as well to determine which are most effective.

3.0.5. *Test with Testing Data.* Finally, we can test our classification on the test data. Once more, we can compute the accuracy and use various k-values to see which yield the highest accuracy.

4. Computational Results

First, we performed the PCA and created a graph comparing the k-value to the cumulative energy to see how the accuracy of our measurements might change with k.

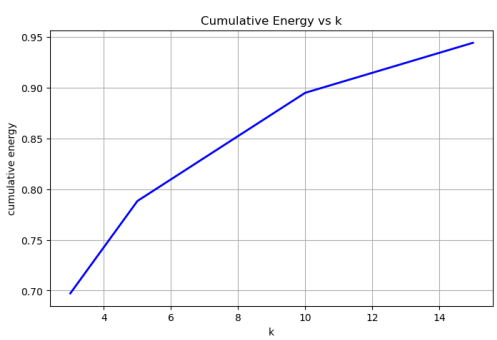

Figure 1. Plot showing the growth of cumulative energy as k increases

As the graph shows, the accuracy of our predictions using PCA increased non-linearly as k increased.

We then visualized the projected training data in two and three dimensions which yielded the following graphs.

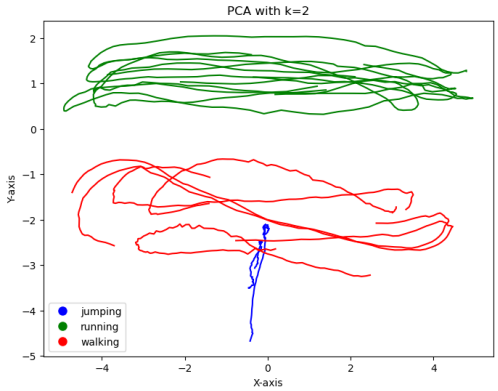

FIGURE 2. 2 dimensional plot of the 15 training samples. The jumping samples are in blue, running in green, and walking in red

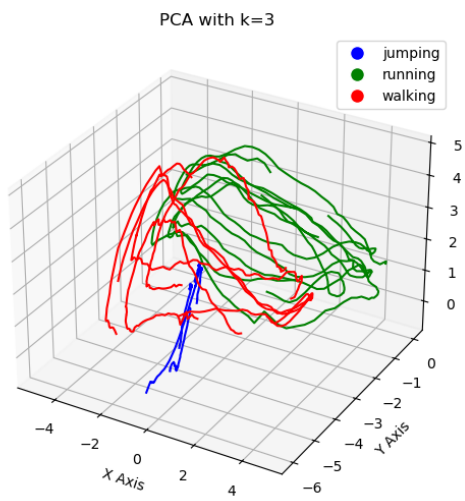

FIGURE 3. 3 dimensional plot of the 15 training samples. The jumping samples are in blue, running in green, and walking in red

From both of these plots, we can begin to see that each of the movements has a different pattern and therefore exists in a different cluster on our plots. This is helpful in thinking about how we can classify future samples.

After applying this classification method, we got the following classifications:

true value	0	0	0	0	0	1	1	1	1	1	2	2	2	2	2
k = 2 - 6	2	0	2	0	0	1	1	1	1	1	2	0	2	2	2
k = 7	2	0	0	0	0	1	1	1	1	1	2	2	2	2	2

TABLE 1. Comparison between the true classifications and the classifications achieved with a few different values of k in the training samples

When k is between 2 and 6, the accuracy is 80 percent, but when the k-value is 7, accuracy increases to 93.3 percent, showing that while keeping only a few of the top k-modes gives us a good chance of correctly classifying movements, increasing the number of k-modes maintained to 7 drastically improves our classification ability.

After seeing how the classification algorithm performed on the training data, we apply it to the testing data. This yields the following results for the classification of the three testing samples:

true value	0	1	2
k = 2	0	2	0
k = 3	0	1	2

Table 2. Comparisons between the true classifications and the classifications achieved with k = 2 and 3 in the testing samples.

With k = 2, our accuracy was only 33.3 percent, which is lower than the 80 percent we got on our training accuracy. However, when we increased k to 3, our algorithm correctly classified each movement yielding 100 percent accuracy. We saw no change between k = 2 and k = 3 in the training data in which the accuracy did not increase until k was brought up to 7.

5. Summary and Conclusions

In conclusion, we developed an algorithm based on principle component analysis to project the movement of a robot, OptimuS-VD, into lower dimensions in order to classify movements as either jumping, running, or walking. This method could now be used to classify the robot's movements in real time with expected accuracy based on the k-value chosen. If we use a k-value between 2 and 6 in the future, we can expect an accuracy of about 80 percent, but if we use a k-value of 7, we can expect an accuracy around 93.3 percent with a chance of higher performance depending on the samples, as shown in the testing data.

References

[1] N. Frank. Extensions of fourier series. AMATH 482, January 26, 2026.
[2] N. Frank. Extensions of fourier series. AMATH 482, January 28, 2026.
[3] N. Frank. Extensions of fourier series. AMATH 482, January 30, 2026.
[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
[5] E. W. Weisstein. "euler angles". *MathWorld–A Wolfram Resource*, 2026.